# State Machines and the LCD Display

| | |
|---|---|
| **Module** | ET4047 |
| **Date** | 28/10/2026 |
| **Lab No.** | Lab 6 |
| **Author** | Luke Griffin |

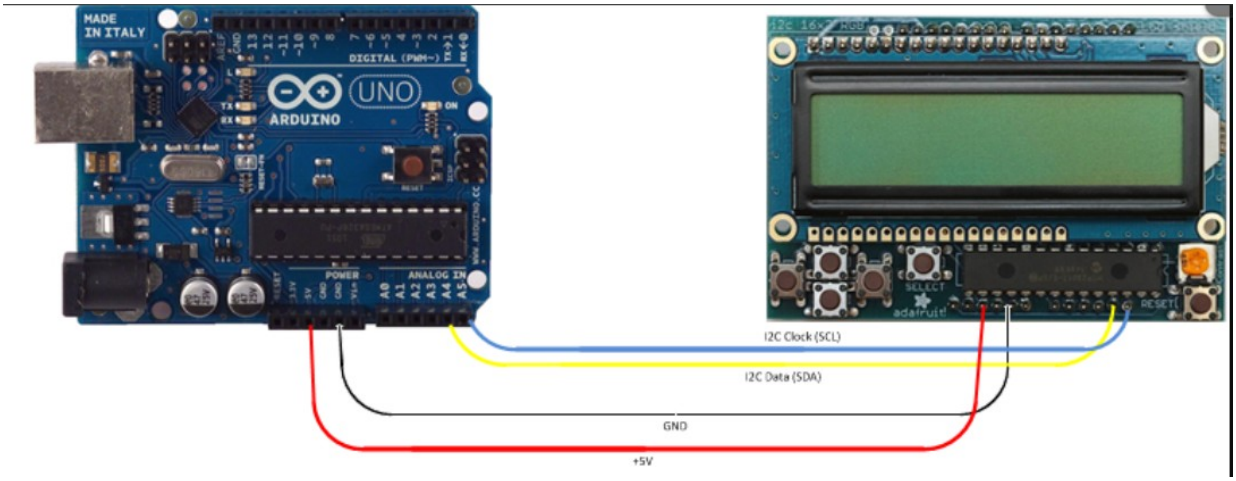



**Figure 1:** Adafruit RGB LCD shield wiring: **5V**, **GND**, **SDA → A4** and **SCL → A5**

## 1 Task 1: A traffic light on the LCD

**Objective:** run a three-state traffic light (RED → GREEN → YELLOW) and change the state machine.

### Wiring

**I2C LCD connections (used in every task)**

| From | | To | Purpose |
|---|---|---|---|
| LCD **GND** | → | Nano **GND** | Ground |
| LCD **VCC** | → | Nano **5V** | Power |
| LCD **SDA** | → | Nano **A4** | I2C data |
| LCD **SCL** | → | Nano **A5** | I2C clock |

© 2026 by University of Limerick. Author: Luke Griffin

### LCD wiring (every task)

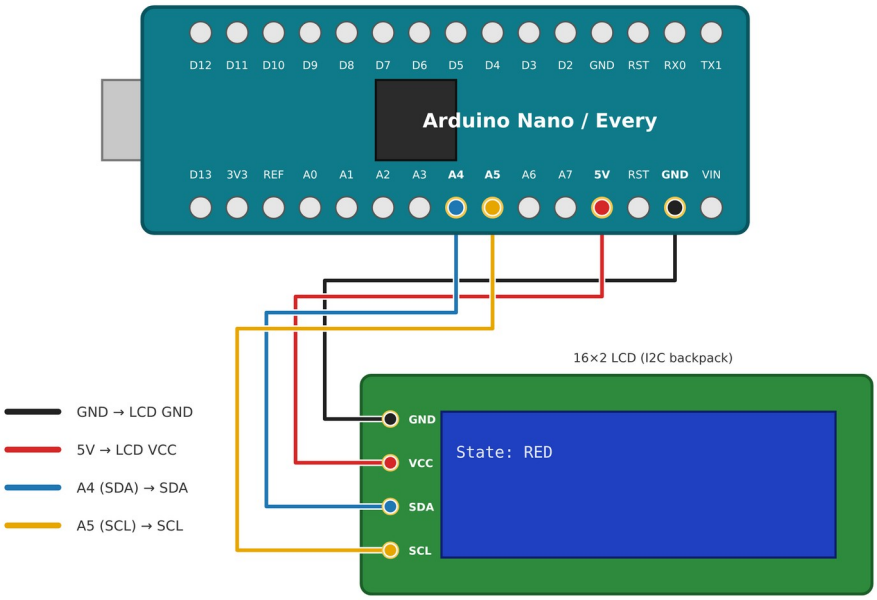



**Figure 2:** I2C backpack LCD wiring. For the RGB LCD shield, see Figure 1.

### Task 1 state diagram: the traffic light

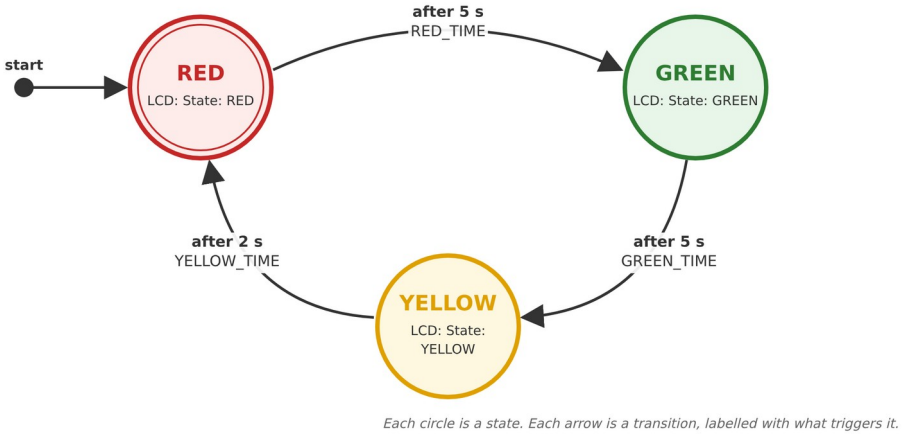



**Figure 3:** Task 1 state diagram

> **Code**
>
> Upload `Task1_Traffic_Light`. Use the folder for your display: `I2C_Backpack_LCD` (separate screen with four wires) or `Adafruit_RGB_LCD_Shield` (the shield with five buttons).

**Expected result:** the LCD shows `State: RED` for 5 s, `State: GREEN` for 5 s, `State: YELLOW` for 2 s, and repeats.

© 2026 by University of Limerick. Author: Luke Griffin

## Task 1 questions

### 1a Run it

Upload the code and check the states change in the right order. **(record)** Take a photo of the LCD.

### 1b The transition table

Fill in the table from the code.

| Current state | Time in state | Next state | LCD shows |
|---|---|---|---|
| RED | | | |
| GREEN | | | |
| YELLOW | | | |

How long does one full cycle take?

*Your answer:*

### 1c Change the timing

Make YELLOW last 3 seconds instead of 2. Which line did you change? Why is it better to keep the times in named constants such as `YELLOW_TIME` than to type `delay(2000)` inside the case?

*Your answer:*

### 1d Add a state

Add a fourth state, `RED_AMBER`, which shows `State: RED+AMBER` for 2 s, so the cycle becomes RED → RED_AMBER → GREEN → YELLOW → RED. Which three places in the code did you change? **(record)** Draw your new state diagram.

*Your answer:*

© 2026 by University of Limerick. Author: Luke Griffin

# 2 Task 2: Traffic light with real LEDs

**Objective:** drive a red, a yellow and a green LED from the state machine, with exactly one LED on at a time.

## Wiring

**LED connections (keep the LCD wiring from Task 1)**

| From | | To | Purpose |
|---|---|---|---|
| Nano **D5** | → | 220 Ω → **red** LED long leg | Red light |
| Nano **D6** | → | 220 Ω → **yellow** LED long leg | Yellow light |
| Nano **D7** | → | 220 Ω → **green** LED long leg | Green light |
| Each LED short leg | → | GND rail → Nano **GND** | Common ground |

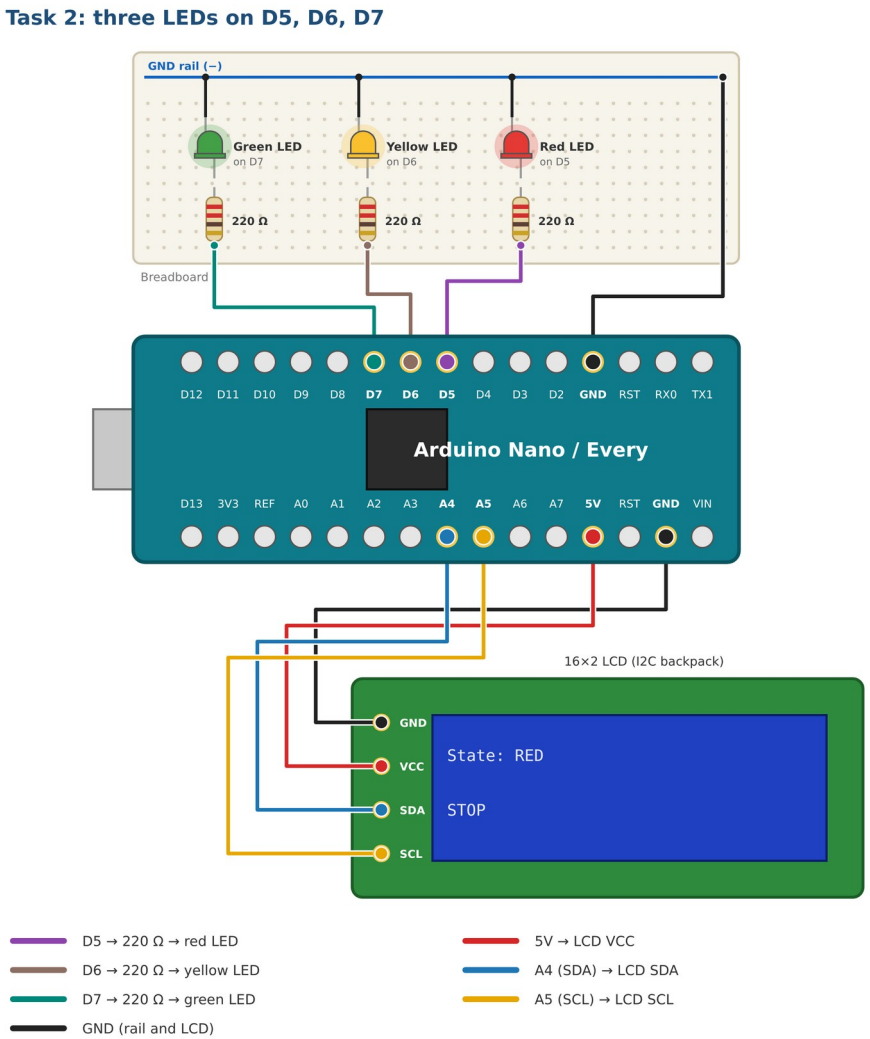



**Figure 4:** Task 2 wiring

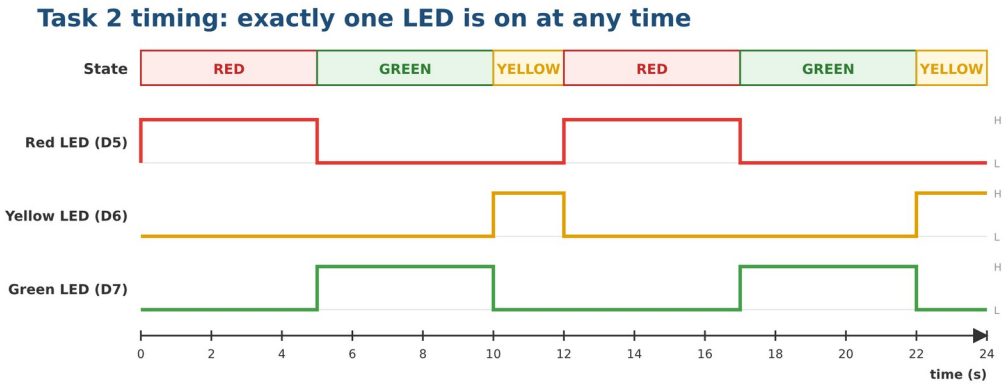



**Figure 5:** Task 2 LED outputs over two cycles

> **Code**
>
> Upload `Task2_Traffic_Light_LEDs`. Use the folder for your display: `I2C_Backpack_LCD` (separate screen with four wires) or `Adafruit_RGB_LCD_Shield` (the shield with five buttons).

**Expected result:** one LED at a time, in step with the LCD (`State: RED` / `STOP`, and so on).

## Task 2 questions

### 2a Run it

**(record)** Paste a few lines from the Serial Monitor. They should look like this:

Serial Monitor

```
Traffic light ready
State 0: RED
State 1: GREEN
State 2: YELLOW
```

### 2b Forget to switch an LED off

In each case, replace the `setLights(…)` line with a single `digitalWrite(…, HIGH)` for that state's LED. Run it for two cycles. What do the LEDs show, and why? Compare with Figure 5. Then put `setLights()` back.

*Your answer:*

### 2c The numbers behind the names

What would the LCD show if you changed the enum to `{ RED, YELLOW, GREEN }` but left the two text arrays as they are? Why?

© 2026 by University of Limerick. Author: Luke Griffin

*Your answer:*

### 2d The resistor

A red LED drops about 2 V and the pin gives 5 V. Roughly how much current flows through the 220 Ω resistor?

*Your answer:*

## 3 Task 3: A pedestrian crossing

**Objective:** the cars' light stays GREEN until the button is pressed and released, then goes YELLOW → WALK (countdown) → CLEAR → GREEN. The sketch uses `millis()` instead of `delay()`, and the button is debounced and reacts on the **rising edge** (when it is released).

### Wiring

**Add the button (keep the LEDs and LCD from Task 2)**

| From | | To | Purpose |
|---|---|---|---|
| Nano **D2** | → | Push button, one leg | Pedestrian button |
| Push button, other leg | → | GND rail | Pressing pulls D2 to 0 V |

© 2026 by University of Limerick. Author: Luke Griffin

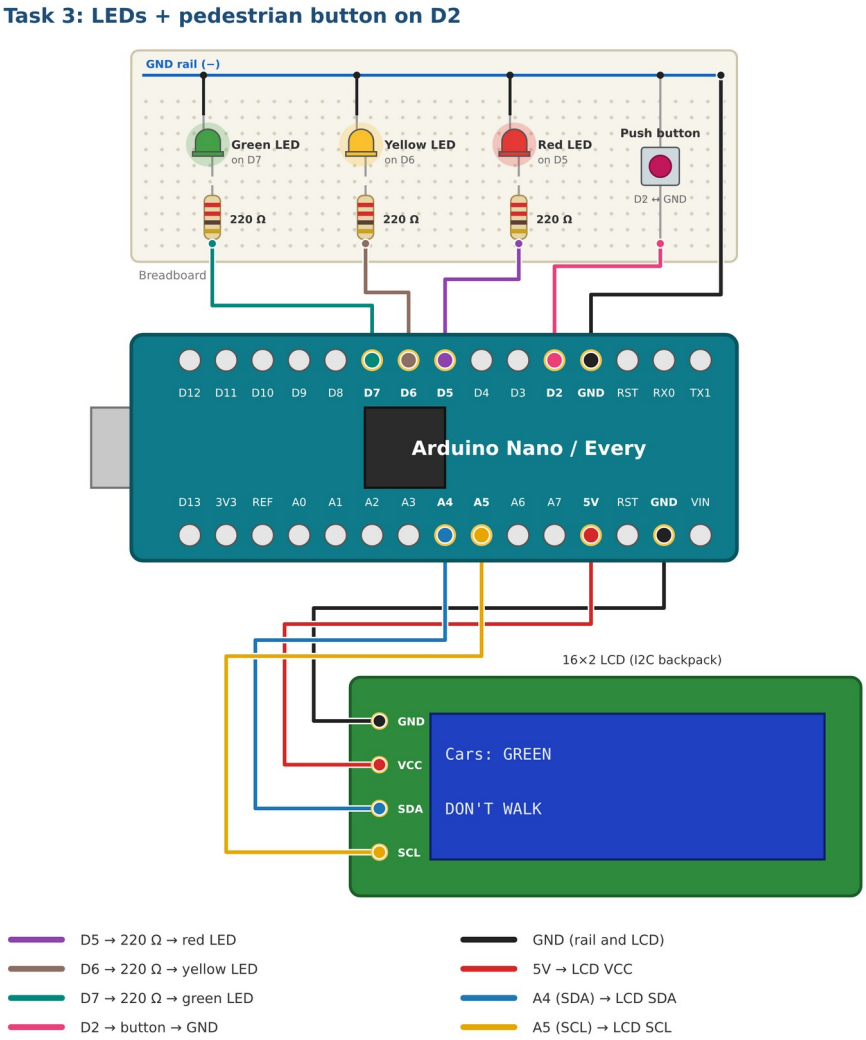



**Figure 6:** Task 3 wiring

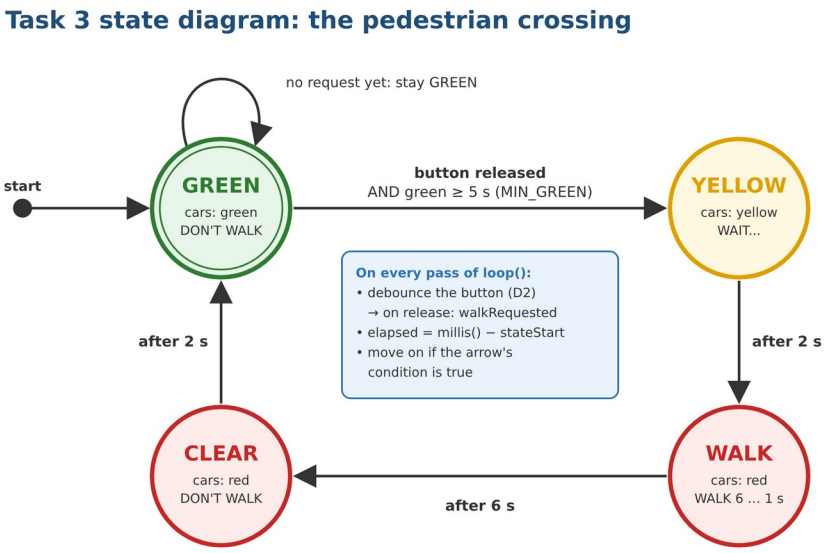



**Figure 7:** Task 3 state diagram

> **Code**
>
> Upload `Task3_Pedestrian_Crossing`. Use the folder for your display: `I2C_Backpack_LCD` (separate screen with four wires) or `Adafruit_RGB_LCD_Shield` (the shield with five buttons).

**Expected result:** the light stays GREEN until you press and release the button; the LCD shows `WAIT...`, then a `WALK 6s … 1s` countdown.

## Task 3 questions

### 3a Run it

Press the button. **(record)** Paste the Serial Monitor output for one crossing. It should look like this:

Serial Monitor

```
Crossing ready
-> GREEN
Button released
-> YELLOW
-> WALK
-> CLEAR
-> GREEN
```

### 3b Press it at different times

Press and release the button at each of the times below. Use the state diagram to explain each result.

| When you press and release | What happens | Why |
|---|---|---|
| Just after the light turns GREEN | | |
| When GREEN has been on for more than 5 s | | |
| During WALK | | |

© 2026 by University of Limerick. Author: Luke Griffin

| When you press and release | What happens | Why |
|---|---|---|
| During CLEAR | | |

### 3c delay() would miss it

If Task 2 checked the button at the top of `loop()`, what is the longest a pedestrian might have to hold it before the board noticed? Why does Task 3 notice a tap of a few milliseconds?

*Your answer:*

### 3d Rising edge and debouncing

1. Press the button and keep holding it. When does `WAIT...` appear: when you press, or when you let go? Which edge of the D2 signal (rising or falling) is the sketch reacting to?

2. In `buttonReleased()`, a new reading only counts once it has stayed the same for `DEBOUNCE_TIME` (50 ms). Why is this needed for a mechanical button?

3. Set `DEBOUNCE_TIME` to `0` and press the button several times. Does anything change? Would it matter if the button started a different action on every press, such as counting pedestrians?

*Your answer:*

© 2026 by University of Limerick. Author: Luke Griffin